2025最新NodeJs面试题

1. 服务端渲染 SSR

服务端渲染是数据与模板组成的 html，即 html=数据+模板。将组件或页面通过服务器生成 html 字符串，再发送到浏览器，最后将静态标记 `混合` 为客户端上完全交互的应用程序。页面没使用服务渲染，当请求页面时，返回的 body 里为空，之后执行 js 将 html 结构注入到 body 里，结合 css 显示出来。

ssr 的优势

- 对 SEO 友好
- 所有的模板、图片等资源都存在服务器端
- 一个 html 返回所有数据
- 减少 http 请求
- 响应快、用户体验好、首屏渲染快

更利于 SEO

不同爬虫工作原理类似，只会爬取源码，不会执行网站的任何脚本(google 除外，据说 google 可以运行 js)。使用 React 或者其他 MVVM 框架之后，页面大多数 DOM 元素都是在客户端根据 js 动态生成，可供爬虫抓取分析的内容大大减少。另外，浏览器爬虫不会等待我们的数据抓取完成之后在抓取页面数据。服务端渲染返回给客户端的是已经获取了异步数据并执行 JS 脚本的最终 html，网络爬虫就可以抓取到完整页面的信息。

更利于首屏渲染

首屏的渲染是 node 发送过来的 html 字符串，并不依赖于 js 文件了，使用客户就会更快的看到页面的内容，尤其是针对大型单页应用，打包后文件体积比较大，普通客户端渲染加载所有所需文件时间较长，首页就会有一个很长的白屏等待时间。

ssr 的局限

- 服务端压力较大

本来是通过客户端完成渲染，现在统一到服务端 node 服务去做。尤其是高并发访问的情况，会大量占用服务端 CPU 资源

- 开发条件受限

在服务端渲染中，react 只会执行到 componentDidMount 之前的生命周期钩子，因此项目引用的第三方的库也不可用其他生命周期钩子，这对引用库的选择产生了很大的限制；

- 学习成本相对较高

除了对 webpack、MVVM 框架要熟悉，还需要掌握 node、koa2 等相关技术。相对于客户端渲染，项目构建、部署过程更为复杂。

时间耗时比较

- 数据请求

由服务端请求首屏数据，而不是客户端请求首屏数据，这就是快的一个主要原因。服务端在内网进行请求，数据响应速度快。客户端在不同网络环境进行数据请求，并且外网 http 请求开销大，导致时间差。

- html 渲染

服务端渲染是先向后端服务器请求数据，然后生成完整首屏 html 返回给浏览器；而客户端渲染时是等 js 代码下载、加载、解析完成之后在请求数据渲染，等待的过程页面是什么都没有的，就是用户所看到的白屏。就是服务端渲染不需要等待 js 代码下载完成并请求数据，就可以返回一个已有完整数据的首屏页面。

2.项目中有没有涉及到 Cluster，说一下你的理解

node 天生就是单线程，在多线程语言对于服务重启以及服务降级，都可以使用其他线程进行监控，当主线程的服务发送服务退出命令后，其他线程就会立即启动进行服务平滑切换和降级，但是 node 却因为天生单线程，所以无法开始多线程去监听服务退出任务。

nodeJS 是单线程运行的，这也是经常被吐槽的点，针对这一点，node 推出了 cluster 这个模块，用于创建多进程的 node 应用。

cluster 可以做下面的事情：

- 发送重启信号给 Master 线程
- 可以根据 cpu 核心数起对应的 n 个新服务并开始监听服务，理论上可以无数个服务，但是一般来说还是根据 os 的核心数去起服务。
- master 线程可以等待旧服务
- 同时还能杀掉旧服务

node 平缓降级与重启的小例子

```
// 一个简单带有cluster自动重启的app.js
const cluster = require("cluster");
const cpuNums = require("os").cpus().length;
const http = require("http");

if (cluster.isMaster) {
  // 是否在主线程
```

```
  for (var i = 0; i < cpuNums; i++) {
    cluster.fork(); // 有多少个cpu就分多少个cluster出来

    cluster.on("exit", function (worker, code, signal) {
      console.log(`线程id为 ${worker.process.pid} 退出`);
    });
    cluster.on("listening", function (worker, code, signal) {
      console.log(`线程id为 ${worker.process.pid} 开始服务`);
    });
    cluster.on("disconnect", function (worker, code, signal) {
      console.log(`线程id为 ${worker.process.pid} 停止服务`);
    });
    process.on("SIGUSR2", function () {
      // 接收kill -SIGUSR2 $pid
      // 保存旧 worker 的列表，cluster.workers 是个 map
      var oldWorkers = Object.keys(cluster.workers).map(function (idx) {
        return cluster.workers[idx];
      });
      // 重新起服务
      cluster.fork();

      // 当新服务起起来之后，关闭所有的旧 worker
      cluster.once("listening", function (worker) {
        oldWorkers.forEach(function (worker) {
          // disconnect 会停止接收新请求，等待旧请求结束后再结束进程
          worker.disconnect();
        });
      });
    });
  }
} else {
  http
    .createServer(function (req, res) {
      res.end(123);
    })
    .listen(8080);

  console.log(`你的线程id为 ${process.pid}`);
}

```

简易的线程关闭自动重启的一个过程。但是工程上面可以使用 pm2 进行服务切换降级，以及对服务更新；我们还可以直接在全局捕错中间件进行 `process.exit()` 事件，进行发送 SIGUSR2 事件，可以自定义启动参数；这个平缓降级主要的两个点：一个是当主线程死掉的时候，正在进来的 request

和正在出去的 response 如何切换，但是这些 pm2 都帮我们做了，另外我们在工程还需要对线程进行自定义的捕错，不然，会遗漏一些不可预知的错误

> Cluster 能给 node 带来性能上很大的提升，让 node 以前让人诟病的单线程也得到解决。同时，Cluster 的多核利用，也能让我们发挥想象力，把它用在其他地方，比如编译打包，比如大数据处理等

3.为什么用 gulp 打包 node?

高效的 gulp

gulp 内部使用的是 node 流机制，流自然就是不需要说了，是一种相当高效且不占内存的一种数据格式，它并不会过多的占用 node 的堆内内存，而且所占内存的最大值在 30m 以内，在流向最后一个消费者才会写入磁盘中， 在打包过程中， 并不会占用磁盘空间或者是内存。

node+gulp 打造高效开发体验面向未来开发

nodejs的痛点---es的支持度，由于 es 版本发布是相当快的，以及一些标准也是很超前的，但是 node 作为运行在服务端的 js 代码，对于这些 es 版本支持的新特性是不会更新这么快的。首当其冲的就是 node 对 es 的支持。在 node 历史长流中，要到 2019 年 11 月 21 日的 13.2 版本才正式支持 ESModule 特效，在 13.2 版本中的 Stability0.1.2 中，在生产模式只能使用 Stability 2 稳定版的 api，而且在这个版本中 nodejs 默认启用对 ES 模块的实验支持，就是说，这里已经是默认允许我们在任何环境使用实验中的 api 了，当然即使这些实验中的 api 不稳定。在以往的 Nodejs8.9.0 之后的版本中需要在启动项目时需要制定特定的参数--experimental-modules，开启对 es 支持以及对实验性 api 支持。

主流浏览器都能通过 `<script type="module">` 标签支持 ECMAScript 模块(ES modules)。各种项目 npm 包都使用了 ES 模块编写，并且可以通过 `<script type="module">` 在浏览器中直接使用。支持导入映射(import maps)即将登陆 Chrome。 import map 将让浏览器支持 node.js 风格的包名导入。基于这个问题，在 node 开发中引入一个中间处理器以支持我们面向未来开发的方式。

打包工具的对比 gulp/Rollup/webpack

首先对于 node 程序的打包要求，这里简单说几点：

1. 在打包后需支持最新的 es 版本
2. 打包编译速度不能太慢，这样对开发进度可控
3. 打包后的文件结构不能发生变化，如有需要可发生一点点变化
4. 配置文件友好

首先第一排除的就是 webpack，因为 webpack 配置起来是比较麻烦的，而且还有很多插件需要版本兼容；在第三点中，打包后文档结构不能发生变化，这样就可以放弃 Rollup 了，因为 Rollup 号称将

所有小文件打包成一个大的 lib 或者是 bin 文件，就是 Rollup 是适合多人共同开发一个库,比较出名的 vue 就是使用 Rollup 打包工具进行打包的。

那么这里选择 gulp 的理由是：

- 第一，编译速度快；
- 第二，开发并不占用很多的电脑内存空间；
- 第三，保持文件结构不变，同时 gulp 还有 Rollup 同样的功能，可以生成 cjs 或者是 mjs 格式的 js 文件。

4.说一下 koa-body 原理

原理

koa-body中间件作用是将 Post 等请求的请求体携带的数据解析到ctx.request.body中。基本原理是先利用type-is--- ctx.js 函数根据请求的 content-type，判断出请求的数据类型，然后根据不同类型用co-body(请求体解析)和 formidable(数据类型是 multipart，文件上传解析)来解析，拿到解析结果以后放到ctx.request.body或者ctx.request.files里面。

使用方式

```
const Koa = require("koa");
const koaBody = require("koa-body");

const app = new Koa();

app.use(koaBody());
app.use((ctx) => {
  ctx.body = `Request Body: ${JSON.stringify(ctx.request.body)}`;
});

app.listen(3000);
```

koa-body 先处理一堆参数，然后用type-is这个包判断请求的数据类型，然后根据不同类型用 co-body 和 formidable 来解析，取到解析结果以后放到 body 或者 files 里面。

- type-is：引用了 mime-types 和 media-typer，但是最终起作用的还是 mime-db 这个包，type-is 就是去匹配请求类型是什么(通过 content-type 来判断，而不是请求内容),我们也可以用 ctx.is 来判断请求的类型。在 type-is 里有一个函数叫 hasbody，get 请求在这个函数的判断下被认为没有 body，所以 get 请求获取到的结果都是空对象。

- co-body：代码结构很简单，提供了 json、form、 text 这三种格式的解析，主要依赖的是 inflation 和 raw-body，处理了一些参数 (主要是 encoding 和 limit 参数)，用 inflation 和 raw-body 解析。inflation 比较简单，根据 content-encoding 的类型，做不同的操作，如果是 gzip 和 deflate，调用 zlib.Unzip 解压缩，如果是 identity，直接返回输入值。raw-body，它做的事情是 stream 解析。
- formidable：如果我们的数据类型是被 type-is 判断为 multipart，那么就会调用 formidable 来进行解析，formidable 本身也提供了很多种格式的解析，有 json，multipart，urlencoded 等 。

---

5. master 挂了的话，pm2 如何处理?

Node.js 原生集群模式

Node.js 提供了 `集群模块` ，简单讲就是复制一些可以共享 TCP 连接的工作线程。

工作原理

集群模块会创建一个 `master 主线程` ，然后复制任意多份程序并启动，这叫做 `工作线程` 。

工作线程通过 IPC 频道进行通信并且使用了 Round-robin algorithm 算法进行工作调度以此实现负载均衡。

Round-robin 调度策略主要是 master 主线程负责接收所有的连接并派发给下面的各个工作线程。

代码的例子：

```
var cluster = require("cluster");
var http = require("http");
var os = require("os");

核数
var numCPUs = os.cpus().length;

if (cluster.isMaster) {
  // Master:
  // Let's fork as many workers as you have CPU cores
  // 创建多个工作线程
  for (var i = 0; i < numCPUs; ++i) {
    cluster.fork();
  }
} else {
  // Worker:
  // Let's spawn a HTTP server
  // (Workers can share any TCP connection.
  //  In this case its a HTTP server)

```

```
  http
    .createServer(function (req, res) {
      res.writeHead(200);
      res.end("hello world");
    })
    .listen(8080);
}

```

你可以不受 CPU 核心限制的创建任意多个工作线程。

用原生方法有些麻烦而且还需要处理，如果某个工作线程挂掉了等额外的逻辑。

pm2 的方式

pm2 内置了处理上述的逻辑，不用再写这么多繁琐的代码了

```
pm2 start app.js -i 4

-i 表示实例程序的个数---就是工作线程。如果i为0表示，会根据当前CPU核心数创建

这样的一行代码就可以啦!
```

1. 保持程序不中断运行 如果有任何工作线程意外挂掉了，pm2 会立即重启他们，当前你可以在任何时候重启，只需要pm2 restart all
2. 实时调整集群数量 你可以使用命令 `pm2 scale <appName> <n>` 调整你的线程数量，如pm2 scale app +3 会在当前基础上加 3 个工作线程
3. 在生产环境中让程序永不中断 `PM2 reload <appName>` 命令一个接一个的重启工作线程，在新的工作线程启动后才结束老的工作线程。

这种方式可以保持你的 Node 程序始终是运行状态。即使在生产环境下部署了新的代码补。

也可以使用 gracefulReload 命令达到同样的目的，它不会立即结束工作线程，而是通过 IPC 向它发送关闭信号，这样它就可以关闭正在进行的连接，还可以在退出之前执行一些自定义任务，这种方式更优雅。

```
process.on("message", function (msg) {
  if (msg === "shutdown") {
    close_all_connections();
    delete_cache();
    server.close();
    process.exit(0);
  }
```

```
});

```

---

6. 上传文件的 content-type 是什么，node 如何拿到上传的文件内容(不适用第三方插件)，文件内容是一次性传输过去的么?

上传文件的 content-type

> 使用 multipart/form-data

如何拿到上传的文件内容

http 模块的 `createServer(request,response)` 传入请求对象的 request，其实已经实现了 ReadableStream 接口，这个信息流可以被监听或者与其它流进行对接。我们可以监听 data 和 end 事件从而把数据给取出来。

文件的内容不是一次性的传过来，是以流的方式传输

获取到上传文件的内容代码如下：

```
const http = require("http");
let fileData = "";
http.createServer((request, response) => {
    request
      .on("error", (err) => {
        console.error(err);
      })
      .on("data", (chunk) => {
        fileData += chunk;
      })
      .on("end", () => {
        console.log(fileData);
      });
  }).listen(8080);

```

---

7. npm2 和 npm3+ 有什么区别?

- npm2 所有项目依赖是 `嵌套关系` 。而 npm3 为了改进嵌套过多、套路过深的情况，会将所有依赖放在第二层依赖中(所有的依赖只嵌套一次，彼此平行，也就是平铺的结构)

- npm2 依赖安装的时候比较简单，直接按照包依赖的树形结构下载填充本地目录结构，也就是说每个包都会将该包的依赖组织到当前包所在的 node_modules 目录中。npm3 则会对依赖安装进行了改造，采用 `扁平结构` 的思路来组织依赖包的目录结构。具体的就是 `npm install` 的过程时：**按照 package.json 里依赖的顺序依次解析，遇到新的包就把它放在第一级目录，后面如果遇到一级目录已经存在的包，会先判断该版本，如果版本一样则忽略，否则会按照npm2的方式依次挂在依赖包目录下。**

8.介绍下 pm2，pm2 依据什么重启服务？

pm2 是一个带有负载均衡功能的 node 应用的进程管理器。我们都知道 node.js 是单进程执行的，当程序出现错误死掉之后需要能够自动，这时候就需要 pm2 了，当然进程管理工具有很多，例如 forever 等等；

`主要特性`

- 启动多子进程，充分使用 CPU
- 子进程之间负载均衡
- 0 秒重启
- 界面友好
- 提供进程交互接口

> 依据什么重启服务
>
> pm2 采用 `心跳检测` 查看子进程是否处于活跃状态，每隔数秒向子进程发送心跳包，子进程如果不回复，那么调用 kill 杀死这个进程，然后再重新 `cluster.fork()` 一个新的进程，子进程可以监听到错误事件，这个时候可以发送消息给主进程，请求杀死自己，并且主进程此时重新调用 cluster.fork() 一个新的子进程

`拥有的能力`

- 日志管理：两种日志，pm2 系统日志与管理的进程日志，默认会把进程的控制台输出记录到日志
- 负载均衡：pm2 可以通过创建共享同一服务器端口的多个子进程来扩展你的应用程序。这样做还允许以零秒停机时间重新启动应用程序
- 终端监控：可以在终端中监控应用程序并检查应用程序运行状况(CPU 使用率、使用的内存、请求/分钟等等)
- SSH 部署：自动部署，避免逐个在所有服务器中进行 ssh
- 静态服务：支持静态服务器功能
- 支持开发调试模式：非后台运行，pm2-dev start `<appName>`

`常用命令`

- 启动服务

```
pm2 start <script_file|config_file> [options] 启动指定应用
```

- 启动一个 node 程序

```
pm2 start app.js 启动 app.js 应用
```

- 启动进程并指定应用的程序名

```
pm2 start app.js --name 程序名启动应用并设置 name
```

- 添加进程监视 监听模式启动，当文件发生变化，自动重启

```
pm2 start app.js --name 程序名 --watch (指定程序名的情况下)

pm2 start app.js --watch (未指定程序名的情况下)
```

- 列出所有进程

```
pm2 list 简写成 pm2 ls
```

- 从进程列表中删除进程

```
pm2 delete `[appname] | id`

pm2 delete app 指定进程名删除

pm2 delete 0 指定进程 id 删除

如果修改了应用配置行为，需要先删除应用，重新启动后方才会生效,如修改脚本入口文件
```

- 删除进程列表中所有进程

```
pm2 delete all (关闭并删除应用)
```

- 查看某个进程具体情况

```
pm2 describe app
```

- 查看进程的资源消耗情况

```
pm2 monit (监控各个应用进程 cpu 和 memory 使用情况)
```

- 重启进程

```
pm2 restart app.js 同时杀死并重启所有进程，短时间内服务不可用，生成环境后慎用

pm2 restart all 重启所有进程

pm2 reload app.js 重新启动所有进行程，0 秒重启，始终保持至少一个进程在运行

pm2 gracefulReload all 以群集横式重新加载所有应用程序

```

- 查看进程日志

```
pm2 log3 [Name] 根据指定应用名查看应用志

pm2 logs [ID] 根据指定应用 ID 查看应用日志

pm2 logs all 查看所有进程的日志
```

- 显示应用程序详细信息

```
pm2 show <appName>[options] 显示指定应用详情

pm2 show [Name] 根据 name 查看

pm2 show [ID] 根据 id 查看
```

- 停止指定应用

```
pm2 stop <appName> [options] 停止指定应用
```

- pm2 stop all 停止所有应用

```
pm2 stop [AppName] 根据应用名停止指定应用

pm2 stop [ID] 根据应用 id 停止指定应用

pm2 kill 杀掉 pm2 管理的所有进程
```

- 启动静态服务器

```
pm2 serve ./dist 8080 将目录 dist 作为静态服务器根目录，端口为 8080
```

- 集群模式启动

```
-i 表示 number-instances 实例数量

max 表示 pm2 将自动检测可用 CPU 的数量 可以自己指定数量

pm2 start app.js -i max 启用群集模式(自动负载均衡)

pm2-dev start … 开发模式启动，即不启用后台运行
```

- 设置 pm2 开机自启

```
开启启动设置，此处是 CentOS 系统
其他系统替换最后一个选项(可选项: ubuntu,centos, redhat, gentoo, systemd, darwin,amazon )

pm2 startup centos 然后按照提示需要输入的命令进行输入

最后保存设置 pm2 save
```

9.说一下koa2和express区别?

概念

`express` 是一个基于Node.js平台的极简、灵活的web应用开发框架，主要基于Connect中间件，并且自身封装了路由、视图处理等功能，使用人数众多

`koa` 相对于更为年轻，是express原班人马基于es新特性重新开发的框架，主要基于co中间件，基于es6 generator 特性的异步流程控制，解决了回调地狱问题和麻烦的错误处理。 `koa框架自身不包含任何中间件` ，很多功能需要借助第三方中间件解决。 `koa2是koa的2.0版本，使用async和await来实现异步流程控制`

区别

1. express自身集成了路由、视图处理等功能；koa本身不集成任何中间件，需要配合路由、视图等中间件进行开发
2. 异步流程控制：express采用callback来处理异步，koa是采用generator，koa2采用async/await。generator和async/await使同步的写法处理异步问题，明显好过callback和promise，async/awaie在语义化上比generator更强。
3. 错误处理：express使用callback捕获异常，对于深层次的异常捕获不了；koa使用try catch，能更好的解决异常捕获
4. 中间件模型： `express基于connect中间件，线性模型` ； `koa中间件采用洋葱模型，所有的请求在经过中间件的时候都会执行两次，能够非常方便的执行一些后置处理逻辑`
5. `context` ：和express只有request和response两个对象不同，koa增加了一个context对象，作为这次请求的上下文对象(在koa中为中间件的this，在koa2中作为中间件的第一个参数传入)。同时context上挂载了request和response两个对象。和express类似，这个对象都提供了大量的便捷方法辅助开发

10.说说使用过 koa2 的哪些中间件，其原理是什么?

常用的中间件

- `koa-router`

路由是 Web 框架必不可少的基础功能，koa.js 为了保持自身的精简，并没有像 Express.js 自带路由功能，因此 koa-router 做了很好的补充，作为 koa 星数最多的中间件, koa-router 提供了全面的路由功能，比如类似 Express 的 app.get/post/put 的写法，URL 命名参数、路由命名、支持加载多个中间件、嵌套路由等。

- `koa-bodyparser`

koa.js 并没有内置 Request Body 的解析器，当我们需要解析请求体时需要加载额外的中间件，官方提供的 koa-bodyparser 是个很不错的选择，支持 x-www-form-urlencoded, application/json 等格式的请求体，但不支持 form-data 的请求体，需要借助 formidable 这个库，也可以直接使用 koa-body 或 koa-better-body

- `koa-views`

koa-views 对需要进行视图模板渲染的应用是个不可缺少的中间件，支持 ejs, nunjucks 等众多模板引擎。

- `koa-static`

Node.js 除了处理动态请求，也可以用作类似 Nginx 的静态文件服务，在本地开发时特别方便，可用于加载前端文件或后端 Fake 数据，可结合 koa-compress 和 koa-mount 使用。

- `koa-session`

HTTP 是无状态协议，为了保持用户状态，我们一般使用 Session 会话，koa-session 提供了这样的功能，既支持将会话信息存储在本地 Cookie，也支持存储在如 Redis,MongoDB 这样的外部存储设备。

- `koa-compress`

当响应体比较大时，我们一般会启用类似 Gzip 的压缩技术减少传输内容，koa-compress 提供 了这样的功能，可根据需要进行灵活的配置。

- `koa-logger`

koa-logger 提供了输出请求日志的功能，包括请求的 url、状态码、响应时间、响应体大小等信息，对于调试和跟踪应用程序特别有帮助，koa-bunyan-logger 提供了更丰富的功能。

- `koa-static`

静态资源服务

中间件原理

Koa 最主要的核心是 `中间件机制洋葱模型`

通过 use()注册多个中间件放入数组中，然后从外层开始往内执行，遇到 next()后进入下一个中间件，当所有中间件执行完后，开始返回，依次执行中间件中未执行的部分，这个整体流程就是递归处理。

```
function compose(middleware) {
  return () => {
    // 先执行第一个函数
    return dispatch(0);

    function dispatch(i) {
      let fn = middleware[i];
      // 如何不存在直接返回 Promise
      if (!fn) {
        return Promise.resolve();
```

```
        }
        // step1: 返回一个 Promise，因此单纯变成一个 Promise 且 立即执行
        // step2: 往当前中间件传入一个next()方法，当这个中间件有执行 next 的时候才执行下一个中间件
        return Promise.resolve(
          fn(function next() {
            // 执行下一个中间件
            return dispatch(i + 1);
          })
        );
      }
    };
  }

```

核心代码是 `return Promise.resolve(fn(context, dispatch.bind(nu1l, i+1)))` ，递归遍历，直到遍历完所有的中间件 next，生成一个多层嵌套的 promise 函数。

koa 的中间件处理可以当做是洋葱模型。中间件数组中中间件的执行是通过递归的方式来执行，调用 dispatch 函数，从第一个开始执行，当有 next 方法时创建一个 promise ，等到下一个中间件执行结果后再执行 next 后面代码。当第二个中间件也有 next 方法时，依然会创建一个新的 promise 等待下一个中间件的执行结果，这也就是中间件 next()的执行原理

app.use() 将中间件 push 到中间件数组中，然后在 listen 方法中通过调用 compose 方法进行集中处理。

11.node 原生 api 错误处理有了解的么？说一下

nodeJS 应用程序一般会遇到以下四类错误：

- 标准的JS错误： 例如 `<EvalError>`、`<SyntaxError>`、`<RangeError>` 、`<ReferenceError>`、`<TypeError>` 或`<URIError>`
- 由底层操作系统触发的系统错误，例如试图打开不存在的文件或者试图更新已关闭的 socket 发送数据
- 由应用程序代码触发的用户自定义的错误
- AssertionError 错误，当 Node.js 检测到不应该发生的异常逻辑时触发。这类错误通常来自 `assert模块`

所有由 nodeJS 引起的 JavaScript 错误与系统错误都继承或实例化标准的 JavaScript `<Error>类` ，并保证至少提供类中的属性

nodeJS 支持几种当应用程序运行时发生的错误的冒泡和处理的机制，如何报告和处理这些错误完全取决于 Error 的类型和被调用的 API 的风格

所有 JavaScript 错误都会被作为异常处理，异常会立即产生并使用标准的 JavaScript `throw` 机制抛出一个错误。这些都是使用 JS 语言提供的 `try...catch...` 语句处理的。

JS 的 `throw机制` 的任何使用都会引起异常，异常必须使用 `try...catch...` 处理，否则 nodeJS 进程会立即退出。

除了少数例外，同步的 API(任何不接受 callback 函数的阻塞方法)会使用 throw 报错

异步的 API 中发生的错误可能会以多种方式进行报告：

- 大多数的异步方法都接受一个 callback 函数，该函数会接受个 Error 对象传入作为第一个参数。 如果第一个参数不是 null 而是一个 Error 实例，则说明发生了错误，应该进行处理。
- 当一个异步方法被一个 EventEmitter 对象调用时，错误会被分发到对象的 `error事件` 上。
- NodeJS API 中有一小部分普通的异步方法仍可能使用 throw 机制抛出异常，且必须使用 `try...catch...` 处理。

12. node 中进程间是如何进行通信的?

进程间通信分类

每个进程都有各自不同的用户地址空间，任何一个进程的全局变量在另一个进程中看不到，所以 `进程之间要交换数据必须通过内核` ，在内核中开辟一个缓存区，进程 A 把数据从用户空间拷贝到内核缓存区，进程 B 再从该缓冲区把数据读走，内核提供的这种机制称为进程间通信

进程间通信(IPC)的方式

- 匿名通道

```
管道是一种半双工的通信方式，数据只能单向流动，而且只能在具有亲缘关系的进程间调用。进程的亲缘关系通常是指父子进程关系
```

- 命名管道

```
命名管道也是半双工的通信方式，但是它允许无亲缘关系进程间通信
```

- 信号量

```
信号量是一个计数器，可以用来控制多个进程对共享资源的访问。它常作为一种锁机制，防止某进程正在访问共享资源时，其他进程也访问该资源。因此，主要作为进程间以及同一进程内不同线程之间
```

```
同步的手段。
```

- 消息队列

```
消息队列是由消息的链表，存放在内核中并由消息队列标识符标识。消息队列克服了信号传递信息少、管道只能承载无格式字节流以及缓存区大小受限等缺点。
```

- 信号

```
信号是一种比较复杂的通信方式，用于通知接收进程某个事件已经发生。
```

- 共享内存

```
共享内存就是映射一段能被其他进程所访问的内存，这段共享内存由一个进程创建，但多个进程都可以访问。共享内容是最快的IPC方式，它是针对其他进程间通信方式运行效率低而专门设计的。它往往与其他通信机制，如信号量配合使用，来实现进程间的同步和通信。
```

- 套接字

```
套接字也是一种进程间通信机制，与其他通信机制不同的是，它可用于不同机器间的进程通信
```

从技术上可以划分四种：

- 消息传递---管道、FIFO、消息队列
- 同步---互斥量、条件变量、读写锁等
- 共享内存---匿名的、命名的
- 远程过程调用

上边提了很多实现进程间通信的方式，那么 node 进程间通信方式是以什么为基础的呢？

node 进程间通信方式

nodeIPC 通过通道技术加事件循环方式进行通信，管道技术在 Windows 下命名管道实现。在 `*nix` 系统则有 Unix Domain Socket 实现，提供给我们简单的 message 事件和 send 方法

管道的定义是什么？

```
管道实际上是在内核中开辟一块缓冲区，它有一个读端一个写端，并传给用户程序两个文件描述符，一个指向读端，一个指向写端，然后该缓冲存储不同进程间写入的内容，并供不同进程读取内容，进而达到通信的目的

管道又分为匿名管道和命名管道，匿名管道常见于一个进程fork出一个子进程，只能亲缘进程通信。而命名管道可以让非亲缘进程进行通信
```

其实本质上来说进程间通信是利用内核管理一块内存，不同进程可以读写这块内容，进而可以互相通信。

那么文件描述符又是什么呢?

在 linux 中一切皆文件，linux 会给每个文件分配一个 id，这个 id 就是文件描述符，指针也是文件描述符的一种。这个很好理解，不过我们可以深入说说，一个进程启动后，会在内核空间(虚拟空间的一部分)创建一个 PCB 控制块，PCB 内部有一个文件描述符表，记录着当前进程所有可用的文件描述符(即当前进程所有打开的文件)。系统除了维护文件描述符表外，还需要维护打开文件表(Open file table)和 i-node(i-node table)

文件打开表(Open file table)包含文件偏移量，状态标志，i-node 表指针等信息

i-node 表(i-node table)包括文件类型、文件大小、时间戳、文件锁等信息

文件描述符不是一对一的，它可以:

- 同一进程的不同文件描述符指向同一文件
- 不同进程可以拥有相同的文件描述符---比如 fork 出的子进程拥有父进程一样的文件描述符，或者不同进程打开同一文件
- 不同进程的同一文件描述符也可以指向不同的文件
- 不同进程的不同文件描述符也可以指向同一个文件

---

13. node 起服务如何保证稳定性?

node 服务的稳定性

对于稳定性，可以分为 `开发` 、 `测试` 、 `代码部署` 、 `正式上线` 、 `回归测试` 、 `hotfix处理` 阶段

一、开发过程

开发时代码质量保证

- 规范新建目录的路径及名称、方便团队项目管理
- 使用 eslint 代码检测
- 每个函数体的大小行数规范管理
- git 分支以及 git 提交记录和 git changelog.md 管理

二、测试阶段

主要是分为 sql 接口测试和接口压力测试

1. sql 接口测试，这里通过 sql 的测试用例，从业务原型抽取测试逻辑，设置边界值，重复循环的测试，当然这里也是测试对应的 sql 表是否达到上线标准
2. 接口压测测试验收，主要测的是 TPS、QPS 和系统吞吐量，简单来说就是将测试服务压力提升到 100%，找到瓶颈处，对于不同环境的接口的吞吐量，接口压力降级等，只要使用的 jmeter 测试工具进行；设置好线程数，线程启动时间，压测时间(可以设置超长时间的测试)，

- `QPS：` Queries Per Second 意思是"每秒查询率"，是一台服务器每秒能够相应的查询次数，是一个特定的查询服务器在规定时间内所处理流量多少的衡量标准
- `TPS：` TransanctionPerSecond 意思是"事务数/秒"。它是软件测试结果的测量单位。一个事务是指一个客户机向服务器发送请求然后服务器做出反应的过程。客户机在发送请求时开始计时，收到服务器响应后结束及时，以此来计算使用的时间和完成的事务个数。

TPS 即每秒处理事务数，包括： 用户请求服务器 服务器自己的内部处理 服务器返回给用户

这三个过程，每秒能够完成 N 个这三个过程，TPS 也就是 N

QPS 基本类似于 TPS，但是不相同的是：对于一个页面的一次访问，形成一个 TPS;但一次页面请求，可能产生多次对服务器的请求，服务器对这些请求就可计入"QPS"之中。例如：访问一个页面会请求服务器 3 次，一次放，产生一个"TPS",产生 3 个"QPS"

1. 系统吞吐量

一个系统的吞吐量(承压能力)与 request 对 CPU 的消耗，外部接口、IO 等等紧密关联。单个 request 对 CPU 消耗越高、外部系统接口、IO 影响速度越慢，系统吞吐能力越低，反之越高

系统吞吐量的几个重要参数：

- QPS(TPS): 每秒钟 request/事务 数量
- 并发数：系统同时处理的 request/事务数
- 响应时间：一般取平均响应时间

可以推算出它们之间的关系：QPS(TPS)=并发数/平均响应时间 或者 并发数=QPS*平均响应时间

当我们吞吐不理想时，这时需要降低一个请求的响应时间，就算是毫秒级的处理也会给服务器性能带来十分大的影响

三、部署阶段

- 网络方面网络均衡方面选用 nginx 对我们的服务进行反向代理以及负载均衡
- 部署方式：分布式部署不同节点的服务器
- 部署容器：使用 docker+k8s 搭建 node 服务运行环境，在大吞吐量下可做到伸缩扩容
- 多 node 服务器下，使用中心指挥官，对 node 服务快速部署启动进行管理，可选用 zookeeper 和携程的 Apollo 进行 node 集群管理

四、正式上线

上线准备主要是日志服务的运行稳定，因为很多报错信息以及服务复盘资料的来源，完善的日志服务和日志打点是完善一个服务很好的工具

五、hotfix

hotfix：是系统出现了问题，需要紧急处理的一个方案；下面就是对于 master 处理的流程，如果经过了 5 次修复依然存在 bug，则将出现问题的模块剔除出来，作为下一次迭代任务的前置任务，用在下一次上线

在服务监控方面 错误日志的监控 心跳监控 监控预警系统 。。。

上面只是简单的叙述了 node 服务稳定运行的一部分基础措施，还有很多工作需要根据时间情况再去做方案

14.node 接口转发有无做什么优化?

- api 削减，可去除后端接口无用的数据
- 做缓存
- 接口代理，解决跨域问题

常见的错误出现在 http 请求层，所以我们可以利用 koa2 next()的核心特性来编写 5xx 和 4xx 的错误处理，这样至少用户不会看到错误的页面。同时我们可以对 Request 等专用请求库进行二次封装，来最小化的降低错误出现的概率。同时一旦出现错误要及时采用 log4js 进行日志记录。最后我们也可以使用全局错误监听 uncaughtExcetion 进行终极的解决。开发阶段采用 PM2 进程守护工具，出现错误能够达到 0 秒热启动。

---

15. node如何平缓降级，重启?

node 天生就是单线程，在多线程语言对于服务重启以及服务降级，都可以使用其他线程进行监控，当主线程的服务发送服务退出命令后，其他线程就会立即启动进行服务平滑切换和降级，但是 node 却因为天生单线程，所以无法开启多线程去监听服务退出任务。

然而 node 本身的 cluster 模块是专门管理这个任务的。 cluster 可以做下面的事情：

1. 发送重启信号给 Master 线程
2. 可以根据 cpu 核心数起对应的 n 个新服务并开始监听服务，理论上可以起无数个服务，但是，一般来说还是根据 os 的核心数去起服务
3. master 线程可以等待旧服务
4. 同时还能杀掉旧服务

```
// 一个简单带有cluster自动重启的app.js
const cluster = require("cluster");
const cpuNums = require("os").cpus().length;
const http = require("http");

if (cluster.isMaster) {
  // 是否在主线程
  for (var i = 0; i < cpuNums; i++) {
    cluster.fork(); // 有多少个cpu就分多少个cluster出来

    cluster.on("exit", function (worker, code, signal) {
      console.log(`线程id为 ${worker.process.pid} 退出`);
    });
    cluster.on("listening", function (worker, code, signal) {
      console.log(`线程id为 ${worker.process.pid} 开始服务`);
    });
    cluster.on("disconnect", function (worker, code, signal) {
      console.log(`线程id为 ${worker.process.pid} 停止服务`);
    });
    process.on("SIGUSR2", function () {
      // 接收kill -SIGUSR2 $pid
      // 保存旧 worker 的列表，cluster.workers 是个 map
      var oldWorkers = Object.keys(cluster.workers).map(function (idx) {
        return cluster.workers[idx];
      });
      // 重新起服务
      cluster.fork();

      // 当新服务起起来之后，关闭所有的旧 worker
      cluster.once("listening", function (worker) {
        oldWorkers.forEach(function (worker) {
          // disconnect 会停止接收新请求，等待旧请求结束后再结束进程
          worker.disconnect();
        });
      });
    });
  }
} else {
  http
    .createServer(function (req, res) {
      res.end(123);
    })
    .listen(8080);

  console.log(`你的线程id为 ${process.pid}`);
}

```

简单的线程关闭、自动重启的一个过程，但是工程上面可以使 pm2 进行服务切换降级以及对服务进行更新；

我们还可以接在全局捕错中间件进行 process.exit() 事件，进行发送 SIGUSR2 事件，可以自定义启动参数

> 这个平缓降级主要的两个点：一个是当主线程死掉的时候，`正在进来的 request` 和 `正在出去的 response` 如何切换，但是这些 `pm2` 都帮我们做了，另外我们在工程还需要对线程进行自定义的捕错，不然，会遗落一些不可预知的错误。

---

16. node 的适用场景以及优缺点是什么？

node.js 是一个基于 Chrome V8 引擎的 JavaScript 运行时

node.js 使用一个事件驱动、非阻塞 I/O 模型，使其轻量、高效、让并发编程更简单、适用于以网络编程为主的 I/O 密集型应用。

自上而下介绍：最底层的是 node.js 依赖的各种库，Chrome V8 解释并执行 JS 代码。Libuv 提供的事件循环、线程池管理、异步网络 I/O 、文件系统 I/O 等能力、负责 I/O 任务的分布和执行，c-ares(DNS 解析)、crypto、http、zlib(压缩)等等，提供了对系统底层工程的访问，如网络、加密、压缩等。

中间是桥接层，链接 JS 和 C/C++的桥梁。Bingdings 把底层 Node.js 可信依赖库暴露的 C/C++库接口转接给 JS 环境。Addons 用于 C/C++扩展。

最上层的应用层，可以调用 Node.js 的各种 api

适用场景

1、RESTful API

这是 Node.js 最理想的应用场景，可以处理数万条连接，本身没有太多的逻辑，只需要请求 API，组织数据进行返回即可。它本质上只是从某个数据库中查找一些值并将它们组成一个响应。由于响应式少量文本，入站请求也是少量的文本，因此流量不高，一台机器甚至也可以处理最繁忙的公司的 API 需求。

2、统一 Web 应用的 UI 层

目前 MVC 的架构，在某种意义上来说，Web 开发有两个 UI 层，一个在浏览器里面我们最终看到的，另一个在 serve 端，负责生成和拼接页面。不讨论这种架构是好是坏，但是有另外一种实践，面向服务的架构，更好的做前后端的依赖分离。如果所有的关键业务逻辑都封装成 REST 调用，就意味着在上层只需要考虑如何用这些 REST 接口构建具体的应用。那些后端程序员们根本不操心具体数据是如何从一个页面传递到另一个页面的，他们也不管用户数据更新是通过 ajax 异步获取的还是通过刷新页面。

3、大量 ajax 请求的应用

例如个性化应用，每个用户看到的页面都不一样，缓存失效，需要在页面加载的时候发起 ajax 请求，nodeJS 能响应大量的并发请求

总而言之，NodeJS 适合运用在 `高并发` 、 `I/O 密集` 、 `少量业务逻辑` 的场景：

```
用户表单收集
考试系统
聊天室
web论坛
图文直播
```

nodeJS 能实现几乎一切的应用，NodeJS 适用在高并发、I/O 密集、少量业务逻辑的场景，我们考虑的点只是适不适用它来做。

优点

1. 是 JS 运行环境，让 JS 也可以开发后端程序
2. 事件驱动。通过单线程维护事件循环队列，没有多线程的资源占用和上下文切换，高效可扩展性强，能充分利用系统资源。
3. 非阻塞 I/O，能处理高并发
4. 单线程(主线程为是单线程)
5. 可伸缩
6. 跨平台，可以应用在 pc Web 端、PC 客户端 nw.js/electron、移动端 cordova、HTML5、react-native、weex、硬件 ruff.io 等。
7. npm 上的各种包模块
8. 配合前端做接口转发 合并请求 消减 JSON 大小 可以独立控制路由(做 SSR 同构) 让前端更有主动性 可以独立部署上线。

缺点

1. 不适合 cpu 密集型应用 CPU 密集型应用给 node 带来的挑战主要是： `由于 JS 单线程的原因，如果有长时间运行的计算将会导致 CPU 时间片不能释放，使得后续 I/O 无法发起` ；解决方案是：分解大型运算任务为多个小任务，使得运算能够适时释放，不能阻塞 I/O 调用的发起；
2. 不适用大内存的应用， `V8 的内存管理机制限制(64 位约 1.4G,32 位约 0.7G)`
3. 不适用大量同步的应用
4. 只支持单核 CPU,不能充分利用 CPU
5. 可靠性低

一旦代码某个环节崩溃，整个系统都崩溃 原因是：单进程、单线程

解决方案：Nginx 反向代理，负载均衡，开多个进程，绑定多个端口；开多个进程监听同一个端口，使用 cluster 模块

1. 开源组件质量参差不齐，更新快，向下不兼容
2. Debug 不方便，错误没有 stack trace

---

17. 介绍下 npm 模块安装机制?输入 npm install 命令敲下回车后它的执行流程是什么?

npm 的概念

npm 是 JS 世界的包管理工具，并且是 Node.js 平台默认的包管理工具。通过 npm 可以安装、共享、分布代码，管理项目依赖关系。

为了在开发过程中，安装功能模块的方便，产生了 npm 包管理器。

在 git clone 项目的时候，项目文件中并没有 node_modules 文件夹，这个文件夹保存的是我们项目开发中所使用的依赖模块。这个文件夹可能几百兆大小，如果放到 github 上，其他人 clone 的时候会非常慢，这个时候需要一个 package.json 依赖配置文件来解决这个问题。

每个人下载这个项目，搭建环境的时候，只需进入该项目目录，直接 npm install ，npm 就会根据这个文件去寻找需要的函数库，也就是依赖。

有些模块是公共开源的，直接写版本号，能直接找到并下载。有些模块是自己研发的，需要写明该包所放置的路径

npm install 执行完之后，我们发现在项目下多了一个 node_modules 文件夹，我们安装的依赖文件都可以在这里找到。

npm install 的执行过程

输入 npm install 命令并敲下回车后，会经历下面的几个阶段:

> 1. 执行工程自身 preinstall

当前 npm 工程如果定义了 preinstall 钩子，此时会被执行

> 1. 确定首层依赖模块

首先需要做的是确定工程中的首层依赖，也就是 dependencies 和 devDependencies 属性中直接指定的模块(假设此时没有添加 npm install 参数)

工程本身是整棵依赖树的根节点，每个首层依赖模块都是根节点下面的一棵子树，npm 会开启多进程从每个首层依赖模块开始逐步寻找更深层级的节点

> 1. 获取模块

获取模块是一个递归的过程，分为以下几步：

`获取模块信息：` 在下载一个模块之前，首先要明确其版本，这是因为 package.json 中往往是 semantic version(语义化版本 semver)。此时如果版本描述文件中有该模块信息直接拿即可，如果没有则从仓库中获取。如 package.json 中某个包的版本是 ^1.1.0，npm 就会去仓库中获取符合 1.1.0 形式的版本

`获取模块实体：` 上一步会获取到模块的压缩包地址(resolved 字段)，npm 会用此地址检查本地缓存，缓存中有就直接拿，如果没有则从仓库中下载。

`查找该模块依赖：` 如果有依赖就回到第一步，如果没有就停止

> 1. 模块扁平化

上一步获取到的是一棵完整的依赖树，其中可能包含大量重复模块。比如 A 模块依赖 lodash，B 模块同样依赖 lodash。在 npm3 以前会严格按照依赖树的结构进行安装，因为会造成模块冗余。

从 npm3 开始默认加入了一个 dedupe 的过程。它会遍历所有的节点，逐个将模块放在根节点下面，也就是 node_modules 的第一层。当发现有重复模块时，则将其丢弃

这里需要对 `重复模块` 进行一个定义，它指的是模块名相同且语义化版本兼容。每个语义化版本都对应一段版本允许范围，如果两个模块的版本允许范围存在交集，那么就可以得到一个兼容版本，而不必版本号完全一致，这可以使更多冗余模块在 dedupe 过程中被去掉。

> 1. 安装模块

这一步将会更新工程中的 node_modules，并执行模块中的生命周期函数(按照 preinstall、install、postinstall 的顺序)

> 1. 执行工程自身的生命周期

当前 npm 工程如果定义钩子此时会被执行(按照 install、postinstall、prepublish、prepare 的顺序)。最后一步是生成或更新版本描述文件，npm install 过程完成

npm 安装模块

- 执行 npm install 命令
- 查询 node_modules 目录之中是否已经存在指定模块

```
若存在，不再重新安装

若不存在
    npm 向registry 查询模块压缩地址
    下载压缩包，存放在根目录下的.npm里
    解压压缩包到当前目录的node_modules目录
```

package-lock.json 建议

`开发系统应用时` ，建议把 package-lock.json 文件提交到代码版本仓库，从而保证所有团队开发者以及 CI 环节可以执行 npm install 时安装的依赖版本都是一致的。

`在开发一个 npm 包时`，你的 npm 包是需要被其他仓库依赖的，由于扁平安装机制，如果锁定了依赖包版本，那你的依赖包就不能和其它依赖包共享语义化版本范围内的依赖包，这样会造成不必要的冗余。所以我们不应该把 package-lock.json 文件发布出去(npm 默认不会把 package-lock.json 文件发布出去)

问: node性能如何优化?

3. web应用层的优化

> 对于web应用主要以下几个提升性能的方法:
>
> 1.动静分离

在普通的web应用中，node尽管也能通过中间件实现静态文件服务，但是 `node处理静态文件的能力并不算突出` 。将图片、字体、样式表和多媒体等静态文件都引导到专业的静态文件服务器上，让node只处理动态请求即可。这个过程可以用 `nginx` 或者专业的 `cdn` 来处理。 将动态请求和静态请求分离后，服务器可以专注在动态服务方面，专业的cdn会将静态文件与用户尽可能靠近，同时能够有更精确和高效的缓存机制。静态文件请求分离后，对动态请求使用不同的域名或多个域名还能消除掉不必要的cookie传输和浏览器对下载线程数的限制。

> 2.启用缓存
>
> 提升性能其实差不多只有两个途径，一个是提升服务的速度，另一个是避免不必要的计算。前者提升的性能在海量流量面前终有瓶颈，但后者却能在访问量越大收益越多。避免不必要的计算，应用场景最多的就是缓存。尽管同步I/O在cpu等待时浪费的时间较为严重，但是在缓存的帮助下，却能消减同步I/O带来的时间浪费。但不管是同步还是异步I/O，避免不必要的计算这条原则入股遵循的好的话，性能提升效果是显著的。
>
> 3.多进程架构
>
> 通过多进程架构，不仅可以充分利用多核cpu，更是可以建立机制能让node进程更加健壮，以保障web应用持续服务。
>
> 4.读写分离

除了动静分离之外，另一个较为重要的分离是读写分离，这主要是针对数据库而言，就任数据库而言， `读取的速度远远高于写入的数据` 。而某些数据库在写入时为了保证数据一致性会进行锁表操作，这同时会影响到读取的速度。某些系统为了提升性能，通常会进行数据库的读写分离，将数据库进行主从设计，这样读数据操作不会受到写入的影响，降低了性能的影响。

4. 代码层的优化

> 1. 使用最新版的nodejs

仅仅使用最新版本的nodejs就能提升性能，nodejs版本的性能提升来自两个方面：

- v8的版本更新
- nodejs内部代码的更新优化

>

> 1. 使用fast-json-stringify加速json序列化

在json序列化时，我们需要识别大量的字段类型。比如对于string类型，我们需要在两边都加下 `"` ,对于数组类型，我们需要遍历数组，把每个对象序列化后，用 `,` 隔开，然后在两边加上 `[ ]`

如果已经提前通过Schema知道每个字段的类型，那么就不需要遍历、识别字段类型，而且可以直接用序列化对应的字段，这就大大减少了计算开销，这就是fast-json-stringify的原理

问: Node 更适合处理 I/O 密集型任务还是 CPU 密集型任务,为什么?

Node 更适合处理 I/O 密集型的任务。

> 因为 Node 的 I/O 密集型任务可以 `异步调用` ，利用事件循环的处理能力，资源占用极少，并且事件循环能力避开了多线程的调用，在调用方面是单线程，内部处理其实是多线程的。
>
> 并且由于 Javascript 是单线程的原因，Node 不适合处理 CPU 密集型的任务，CPU 密集型的任务会导致 `CPU 时间片` 不能释放，使得后续 I/O 无法发起，从而造成阻塞。但是可以利用到多进程的特点完成对一些 CPU 密集型任务的处理，不过由于 Javascript 并不支持多线程，所以在这方面的处理能力会弱于其他多线程语言（例如 Java、Go）

问: node性能如何监控及优化手段?

监控分类

- 一种是 `业务逻辑型` 的监控
- 一种是 `硬件` 的监控

性能监控

node性能监控主要分为以下几点：

1. `日志监控`

可以通过监控异常日志的变动，将新增的异常类型和数量反映出来 监控日志可以实现pv和uv的监控，通过pv/uv的监控，可以知道使用者们的使用习惯，预知访问高峰

1. `响应时间`

响应时间也是一个需要监控的点。一旦系统的某个子系统出现异常或者性能瓶颈将会导致系统的响应时间变长。响应时间可以在nginx一类的反向代理上监控，也可以通过应用自己产生访问日志来监控

1. `进程监控`

监控日志和响应时间都能较好地监控到系统的状态，但是它们的前提是系统是运行状态的，所以监控进程是比前两者更为紧要的任务。监控进程一般是检查操作系统中运行的应用进程数，比如对于采用多进程架构的web应用，就需要检查工作进程的数，如果低于低估值，就应当发出警报

1. `磁盘监控`

磁盘监控主要是监控磁盘的用量。由于写日志频繁的缘故，磁盘空间渐渐被用光。一旦磁盘不够用将会引发系统的各种问题，给磁盘的使用量设置一个上限，一旦磁盘用量超过警戒值，服务器的管理者应该整理日志或者清理磁盘

1. `内存监控`

对于node而言，一旦出现内存泄漏，不是那么容易排查的。监控服务器的内存使用情况。如果内存只升不降，那么铁定存在内存泄漏问题。符合正常的内存使用应该是有升有降，在访问量大的时候上升，在访问量回落的时候，占用量也随之回落。监控内存异常时间也是防止系统出现异常的好方法。如果突然出现内存异常，也能够追踪到近期的哪些代码改动导致的问题

1. `cpu占用监控（CPU使用率）`

服务器的cpu占用监控也是必不可少的项，cpu的使用分为用户态、内核态、IOWait等。如果用户态cpu使用率较高，说明服务器上的应用需要大量的cpu开销；如果内核态cpu使用率较高，说明服务器需要花费大量时间进行进程调度或者系统调用；IOWait使用率反映的是cpu等待磁盘I/O操作；cpu的使用率中，用户态小于70%，内核态小于35%且整体小于70%，处于正常范围。监控cpu占用情况，可以帮助分析应用程序在实际业务中的状况。合理设置监控阈值能够很好地预警

1. `cpu load监控（CPU负载）`

cpu load又称cpu平均负载。它用来`描述操作系统当前的繁忙程度`，又简单地理解为cpu在单位时间内正在使用和等待使用cpu的平均任务数。它有3个指标，即1分钟的平均负载、5分钟的平均负载，15分钟的平均负载。cpu load过高说明进程数量过多，这在node中可能体现在用于进程模块反复启动新的进程。监控该值可以防止意外发生

1. `I/O负载`

I/O负载指的主要是磁盘I/O。反应的是磁盘上的读写情况，对于node编写的应用，主要是面向网络业务，是不太可能出现I/O负载过高的情况，`大多数的I/O压力来自于数据库`。不管node进程是否与数据库或其他I/O密集的应用共同处理相同的服务器，我们都应该监控该值防止意外情况

1. `网络监控`

虽然网络流量监控的优先级没有上述项目那么高，但还是需要对流量进行监控并设置流量上限值。即便应用突然受到用户的青睐，流量暴涨的时候也可以通过数值感知到网站的宣传是否有效。一旦流量超过警戒值，开发者就应当找出流量增长的原因。对于正常增长，应当评估是否该增加硬件设备来为更多用户提供服务。网络流量监控的两个主要指标是流入流量和流出流量

应用状态监控

除了这些硬性需要检测的指标之外，应用还应该提供一种机制来反馈其自身的状态信息，外部监控将会持续性地调用应用地反馈接口来检查它地健康状态。

dns监控

dns是网络应用的基础，在实际的对外服务产品中，多数都对域名有依赖。dns故障导致产品出现大面积影响的事件并不少见。由于dns服务通常是稳定的，容易让人忽略，但是一旦出现故障，就可能是史无前例的故障。对于产品的稳定性，域名dns状态也需要加入监控。

如何监控

关于性能方面的监控，一般情况都需要借助工具来实现

这里采用 `Easy-Monitor 2.0` ，其是轻量级的 Node.js 项目内核性能监控 + 分析工具，在默认模式下，只需要在项目入口文件 require 一次，无需改动任何业务代码即可开启内核级别的性能监控分析

使用方法如下：

在你的项目入口文件中按照如下方式引入，当然请传入你的项目名称：

```
const easyMonitor = require('easy-monitor');
easyMonitor('你的项目名称');
```

如何优化

关于Node的性能优化的方式有：

- 使用最新版本Node.js
- 正确使用流 Stream
- 代码层面优化
- 内存管理优化

使用最新版本Node.js

每个版本的性能提升主要来自于两个方面：

- V8 的版本更新
- Node.js 内部代码的更新优化

> 正确使用流 Stream

在Node中，很多对象都实现了流， `对于一个大文件可以通过流的形式发送，不需要将其完全读入内存`

```
const http = require('http');
const fs = require('fs');

// bad
http.createServer(function (req, res) {
    fs.readFile(__dirname + '/data.txt', function (err, data) {
```

```
        res.end(data);
    });
});

// good
http.createServer(function (req, res) {
    const stream = fs.createReadStream(__dirname + '/data.txt');
    stream.pipe(res);
});

```

代码层面优化

合并查询，将多次查询合并一次，减少数据库的查询次数

```
// bad
for user_id in userIds
     let account = user_account.findOne(user_id)

// good
const user_account_map = {}   // 注意这个对象将会消耗大量内存。
user_account.find(user_id in user_ids).forEach(account){
    user_account_map[account.user_id] =  account
}
for user_id in userIds
    var account = user_account_map[user_id]

```

内存管理优化

在 V8 中，主要将内存分为新生代和老生代两代：

- 新生代：对象的存活时间较短。新生对象或只经过一次垃圾回收的对象
- 老生代：对象存活时间较长。经历过一次或多次垃圾回收的对象

若新生代内存空间不够，直接分配到老生代

通过减少内存占用，可以提高服务器的性能。如果有内存泄露，也会导致大量的对象存储到老生代中，服务器性能会大大降低

如下面情况：

```
const buffer = fs.readFileSync(__dirname + '/source/index.htm');

app.use(
```

```
    mount('/', async (ctx) => {
        ctx.status = 200;
        ctx.type = 'html';
        ctx.body = buffer;
        leak.push(fs.readFileSync(__dirname + '/source/index.htm'));
    })
);

const leak = [];

```

leak的内存非常大，造成内存泄露，应当避免这样的操作，通过减少内存使用，是提高服务性能的手段之一

而 `节省内存最好的方式是使用池` ，其将频用、可复用对象存储起来，减少创建和销毁操作

例如有个图片请求接口，每次请求，都需要用到类。若每次都需要重新new这些类，并不是很合适，在大量请求时，频繁创建和销毁这些类，造成内存抖动

使用对象池的机制，对这种频繁需要创建和销毁的对象保存在一个对象池中。每次用到该对象时，就取对象池空闲的对象，并对它进行初始化操作，从而提高框架的性能

问: 说说你理解的 node 中间层怎样做的请求合并转发?

2. 什么是中间层?

前端 -> nodejs -> 后端 -> nodejs -> 数据处理 -> 前端

这么一个流程，这个流程的好处就是当业务逻辑过多，或者业务需求在不断变更的时候，前端不需要过多当去改变业务逻辑，与后端低耦合。前端即显示，渲染。后端获取和存储数据。中间层处理数据结构，返回给前端可用可渲染的数据结构。

nodejs是起中间层的作用，即根据客户端不同请求来做相应的处理或渲染页面，处理时可以是把获取的数据做简单的处理交由底层java那边做真正的数据持久化或数据更新，也可以是从底层获取数据做简单的处理返回给客户端。

通常我们把Web领域分为客户端和服务端，也就是前端和后端，这里的后端就包含了网关，静态资源，接口，缓存，数据库等。 `而中间层呢，就是在后端这里再抽离一层出来，在业务上处理和客户端衔接更紧密的部分，比如页面渲染（SSR），数据聚合，接口转发等等` 。

以SSR来说，在服务端将页面渲染好，可以加快用户的首屏加载速度，避免请求时白屏，还有利于网站做SEO，他的好处是比较好理解的

3. 中间层可以做的事情

- 代理：在开发环境下，我们可以利用代理来，解决最常见的跨域问题；在线上环境下，我们可以利用代理，转发请求到多个服务端。

- 缓存：缓存其实是更靠近前端的需求，用户的动作触发数据的更新，node中间层可以直接处理一部分缓存需求。
- 限流：node中间层，可以针对接口或者路由做响应的限流。
- 日志：相比其他服务端语言，node中间层的日志记录，能更方便快捷的定位问题（是在浏览器端还是服务端）。
- 监控：擅长高并发的请求处理，做监控也是合适的选项。
- 鉴权：有一个中间层去鉴权，也是一种单一职责的实现。
- 路由：前端更需要掌握页面路由的权限和逻辑。
- 服务端渲染：node中间层的解决方案更灵活，比如SSR、模板直出、利用一些JS库做预渲染等等。

4. node转发API（node中间层）的优势

- 可以在中间层把java|php的数据，处理成对前端更友好的格式
- 可以解决前端的跨域问题，因为服务器端的请求是不涉及跨域的，跨域是浏览器的同源策略导致的
- 可以将多个请求在通过中间层合并，减少前端的请求

5. 如何做请求合并转发

- 使用 `express` 中间件 `multifetch` 可以将请求批量合并
- 使用 `express` + `http-proxy-middleware` 实现接口代理转发

6. 不使用用第三方模块手动实现一个nodejs代理服务器，实现请求合并转发

> 1. 实现思路

```
①搭建http服务器，使用Node的http模块的createServer方法
②接收客户端发送的请求，就是请求报文，请求报文中包括请求行、请求头、请求体
③将请求报文发送到目标服务器，使用http模块的request方法
```

> 1. 实现步骤

第一步：http服务器搭建

```
const http = require("http");
const server = http.createServer();
    server.on('request',(req,res)=>{
    res.end("hello world")
})
server.listen(3000,()=>{
    console.log("running");
})

```

第二步：接收客户端发送到代理服务器的请求报文

```
const http = require("http");
const server = http.createServer();
server.on('request',(req,res)=>{
    // 通过req的data事件和end事件接收客户端发送的数据
    // 并用Buffer.concat处理一下
    let postbody = [];
    req.on("data", chunk => {
        postbody.push(chunk);
    })
    req.on('end', () => {
        let postbodyBuffer = Buffer.concat(postbody);
        res.end(postbodyBuffer)
    })
})
server.listen(3000,()=>{
    console.log("running");
})

这一步主要数据在客户端到服务器端进行传输时在no

```

这一步主要数据在客户端到服务器端进行传输时在nodejs中需要用到buffer来处理一下。处理过程就是将所有接收的数据片段chunk塞到一个数组中，然后将其合并到一起还原出源数据。合并方法需要用到Buffer.concat，这里不能使用加号，加号会隐式的将buffer转化为字符串，这种转化不安全。

第三步：使用http模块的request方法，将请求报文发送到目标服务器

第二步已经得到了客户端上传的数据，但是缺少请求头，所以在这一步根据客户端发送的请求需要构造请求头，然后发送

```
const http = require("http");
const server = http.createServer();

server.on("request", (req, res) => {
    var { connection, host, ...originHeaders } = req.headers;
    var options = {
        "method": req.method,
        // 随表找了一个网站做测试，被代理网站修改这里
        "hostname": "www.nanjingmb.com",
        "port": "80",
        "path": req.url,
```

```
            "headers": { originHeaders }
        }
        //接收客户端发送的数据
        var p = new Promise((resolve,reject)=>{
            let postbody = [];
            req.on("data", chunk => {
                postbody.push(chunk);
            })
            req.on('end', () => {
                let postbodyBuffer = Buffer.concat(postbody);
                resolve(postbodyBuffer)
            })
        });
        //将数据转发，并接收目标服务器返回的数据，然后转发给客户端
        p.then((postbodyBuffer)=>{
            let responsebody=[]
            var request = http.request(options, (response) => {
                response.on('data', (chunk) => {
                    responsebody.push(chunk)
                })
                response.on("end", () => {
                    responsebodyBuffer = Buffer.concat(responsebody)
                    res.end(responsebodyBuffer);
                })
            })
            // 使用request的write方法传递请求体
            request.write(postbodyBuffer)
            // 使用end方法将请求发出去
            request.end();
        })
    });
    server.listen(3000, () => {
        console.log("runnng");
    })

```

问: 请解释一下Node.js的优点是什么?

Node.js的主要优点包括以下几个方面:

1. 异步编程模型：Node.js采用了事件驱动、非阻塞I/O模型，使其能够在大量连接的情况下保持高性能和稳定性。

2. 单一编程语言：Node.js使用的是JavaScript，这是一种被广大开发者所熟悉的语言，能够大大降低学习成本。
3. 开源社区活跃：Node.js有一个非常活跃的开源社区，有大量的模块可供使用，极大地提高了开发效率。
4. 跨平台性：Node.js可以在多种操作系统上运行，如Windows、Linux、macOS等。
5. 高性能：由于Node.js的非阻塞I/O模型，使其在处理大量并发请求时具有极高的性能。
6. 数据密集型应用：Node.js擅长处理数据密集型应用，如实时通信、实时流媒体、实时游戏等。

综上所述，Node.js凭借其独特的异步编程模型和强大的社区支持，成为了一种优秀的服务器端技术选择。

问: Node.js采用哪种编程模式？它的优势是什么？

Node.js采用的是事件驱动编程模式，这种模式使得Node.js能够在不需要等待I/O操作完成的情况下继续执行其他的任务。这样做的好处是可以提高系统的吞吐量，尤其是在处理高并发的网络请求时，能够大大提高响应速度。

此外，Node.js还支持回调函数、Promise和async/await等不同的异步编程方式，可以根据具体的应用场景灵活选择合适的编程方式。这样做的好处是可以提高代码的可读性和可维护性，同时也能够更好地控制代码的执行流程。

总的来说，Node.js的事件驱动编程模式和灵活的异步编程方式为其带来了高效、稳定和易用的优势。

问: 解释一下Event Loop的工作原理。

Event Loop是Node.js实现异步编程的关键部分，它负责管理所有的事件和回调函数。

当一个事件触发时，Event Loop会将相关的回调函数放入队列中等待执行。这些回调函数按照一定的优先级进行排列，一旦执行环境空闲，Event Loop就会开始执行队列中的下一个回调函数。

当一个回调函数执行完毕后，Event Loop会重新检查队列，看是否有新的事件需要处理。如果有，它会将相关的回调函数放入队列中；如果没有，它则会进入等待状态，直到有新的事件发生。

这就是Event Loop的基本工作原理，通过不断地检查和执行回调函数，实现了Node.js的异步编程能力。

问: 你是如何使用npm来管理Node.js项目的依赖关系的？

我通常使用npm来进行Node.js项目的依赖关系管理。

首先，在项目根目录下创建一个名为 `package.json` 的文件，其中包含项目的元信息和依赖列表。然后，我会在这个文件中列出项目所需要的每个依赖及其版本号。比如，如果我的项目需要Express框架，那么我可以在 `dependencies` 对象中添加一行代码如下：

```
"dependencies": {    "express": "^4.17.1"}
```

接着，我可以使用 `npm install` 命令来安装这些依赖项。这个命令会在本地的 `node_modules` 目录下下载并安装指定的依赖包。如果在后续的开发过程中，我又需要用到其他的新依赖包，则可以修改 `package.json` 文件并再次运行 `npm install` 命令。

除此之外，我还常用 `npm update` 命令来更新已有的依赖包到最新版本。在开发完成后，还可以使用 `npm publish` 命令将项目发布到npm registry供他人使用。

总之，npm提供了强大的功能来管理和分发Node.js项目的依赖关系，使得整个开发过程变得更加简单和高效。

问: Node.js的同步和异步编程是什么意思?

Node.js的同步和异步编程是一种描述程序执行方式的不同术语。

同步编程指的是程序按照顺序依次执行的方式。在这种方式下，程序会逐行地执行代码，并且在某条代码执行完毕之前，其他后续代码都会暂停等待。这种方式适用于那些顺序依赖性强、逻辑复杂度低的情况。

而异步编程则是指程序在同一时间内可以并行执行多个任务的方式。在这种方式下，程序可以不按顺序地执行代码，并且可以在某些任务未完成时就开始执行其他任务。这种方式适用于那些顺序依赖性较弱、并发性要求较高的情况。

Node.js主要支持异步编程，并且通过Event Loop机制实现了高效的并发能力。然而，为了兼容其他编程风格，也支持一些同步编程的功能。

问: 如何使用ES6的Promise和async/await来处理异步操作?

在ES6中，Promise是一种用于处理异步操作的对象，它可以代替传统的回调函数，让异步操作更加直观易懂。而async/await是基于Promise的一种新语法糖，可以让异步代码看起来像同步代码一样执行。

下面是使用Promise的一个例子：

```
// 创建一个Promise实例
let promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve('Hello from Promise!');
  }, 2000);
});

// 使用.then方法处理Promise的结果
promise
  .then(result => {
```

```
    console.log(result); // 输出 'Hello from Promise!'
  });

```

下面是使用async/await的例子：

```
// 定义一个异步函数
async function getServerTime() {
  const response = await fetch('https://api.example.com/time'); // 异步操作
  const result = await response.json(); // 另一个异步操作
  console.log(result);
}

// 调用异步函数
getServerTime();

```

在这个例子中，我们定义了一个异步函数 `getServerTime` ，它包含两个异步操作：发送GET请求和解析JSON数据。使用 `async` 关键字声明了这是一个异步函数，使用 `await` 关键字表示要等待这两个异步操作完成后再继续执行。这种方式可以让异步代码看起来更像同步代码，提高了代码的可读性和可维护性。

问: 解释一下Node.js的Express框架及其主要组件。

Express是一个基于Node.js的web应用程序框架，它可以帮助开发人员快速构建Web服务。它的主要组件包括：

1. Router：路由器，用于定义路由规则，根据URL的不同路径映射到不同的处理器。
2. Middleware：中间件，用于拦截HTTP请求，并对其进行预处理或后处理。
3. View engine：视图引擎，用于渲染HTML模板。
4. Body-parser：用于解析POST请求体中的内容，如表单数据和JSON数据。
5. Cors：跨域资源共享模块，允许客户端向不同域名的服务器发起请求。
6. Static：静态文件服务器，用于托管静态文件，如图片、CSS和JavaScript文件。

Express还支持自定义错误处理器、HTTP中间件等功能，使得开发者可以轻松地搭建出高质量的Web服务。

问: 你如何在Node.js中使用中间件来组织代码?

在 Node.js 中使用中间件可以有效地组织代码，使代码更易于理解和维护。中间件是 Node.js 应用程序的一个重要组成部分，它们负责执行异步操作、处理请求和响应等任务。 使用中间件的一般步骤如下：

1. 定义中间件函数。中间件函数接受三个参数：request 对象、response 对象和 next 函数。其中，next 函数用于通知中间件链中的下一个中间件或最终的路由处理器继续执行。
2. 使用 app.use() 方法注册全局中间件。全局中间件对所有路由都有效，通常用于进行身份验证、日志记录等操作。
3. 使用 router.use() 方法注册局部中间件。局部中间件只对特定的路由有效，通常用于处理与该路由相关的业务逻辑。
4. 在应用程序的主文件中引入并启动中间件。在主文件中，需要使用 require() 方法引入中间件文件，并使用 app.listen() 方法启动中间件。

通过以上方法，在 Node.js 中使用中间件可以帮助我们更好地组织代码，提高代码的可读性和可维护性。同时，中间件也可以帮助我们更好地管理和控制应用程序的状态和行为，从而实现更加复杂的功能和需求。

问: 你如何在Node.js中处理HTTP请求和响应?

在 Node.js 中处理 HTTP 请求和响应的基本流程如下：

1. 创建 HTTP 服务端程序。首先需要使用 http 模块创建一个 HTTP 服务端程序，并设置相应的回调函数。
2. 监听 HTTP 请求事件。然后需要使用 http.createServer() 方法创建一个 HTTP 服务实例，并监听 request 事件以接收客户端发送来的 HTTP 请求。
3. 处理 HTTP 请求。当接收到 HTTP 请求时，需要解析请求头、查询字符串以及请求体中的数据，并根据请求的方法和路径来确定具体的响应内容。
4. 设置响应头和状态码。接着需要设置响应头和状态码，并将响应内容写入响应体中。
5. 发送响应到客户端。最后需要使用 response.end() 方法将响应发送回客户端。

通过以上步骤，可以在 Node.js 中有效地处理 HTTP 请求和响应，为用户提供丰富的 Web 功能和服务。 需要注意的是，由于 Node.js 是单线程模型，因此在同一时刻只能处理一个请求。为了能够同时处理多个请求，可以通过集群、负载均衡等方式将多个请求分发到不同的 Node.js 进程中进行处理。

问: 请解释一下Node.js的集群化工作模式?

Node.js 的集群化工作模式是一种利用多个进程同时运行 Node.js 应用来实现负载均衡的技术。它可以充分利用多核 CPU 的计算能力，并且可以减少单个进程的压力，从而提高应用的可用性和性能。 在 Node.js 中，可以使用内置的 cluster 模块来创建多个子进程，每个子进程都可以独立地运行 Node.js 应用。在主进程中，可以使用 cluster.fork() 方法创建一个新的子进程，并将其绑定到一个端口上。当有新的连接请求到达时，Node.js 会根据当前的负载情况将请求分发到一个空闲的子进程中进行处

理。 此外，Node.js 还提供了一些内置的策略来实现负载均衡，如 round-robin 和 least-connections 等。用户可以根据自己的实际需求选择合适的负载均衡策略，以达到最佳的效果。 总之，Node.js 的集群化工作模式可以有效地提高应用的性能和可用性，特别是在处理大量并发请求时尤其有用。

问:什么是错误优先的回调函数?

错误优先的回调函数用于传递错误和数据。第一个参数始终应该是一个错误对象， 用于检查程序是否发生了错误。其余的参数用于传递数据。例如:

```
fs.readFile(filePath, function(err, data) {
    if (err) {
        //handle the error
    }
    // use the data object
});
```

解析：这个题目的主要作用在于检查被面试者对于Node中异步操作的一些基本知识的掌握。

问:如何避免回调地狱

你可以有如下几个方法:

- 模块化：将回调函数分割为独立的函数
- 使用Promises
- 使用yield来计算生成器或Promise

解析：这个问题有很多种答案，取决你使用的场景，例如ES6, ES7，或者一些控制流库。

问:在Node中你如何监听80端口

这题有陷阱！在类Unix系统中你不应该尝试监听80端口，因为这需要超级用户权限， 因此不建议让你的应用监听这个端口。

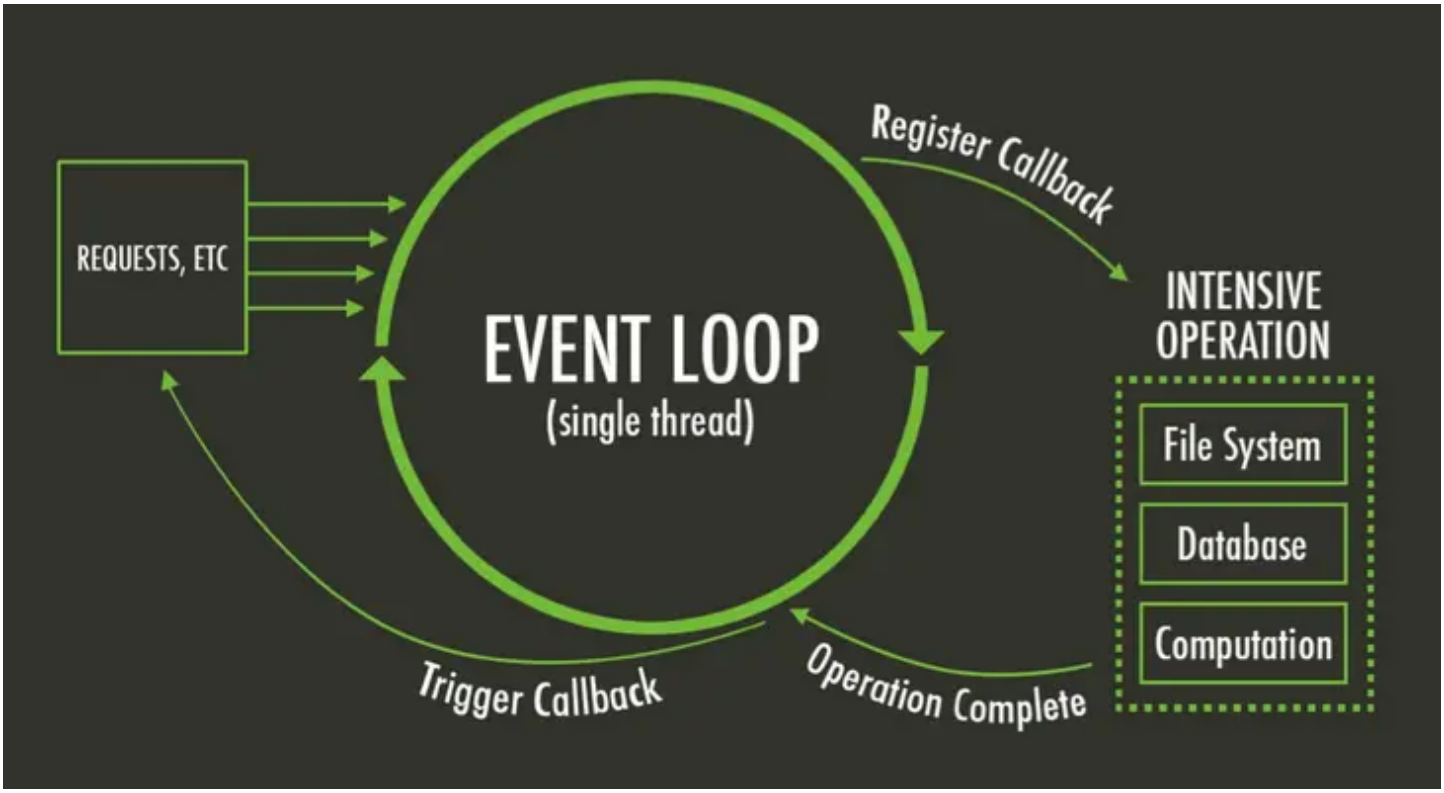
目前，如果你想让你的应用一定要监听80端口，可以这么做：让你的Node应用监听大于1024的端口， 然后在它前面在使用一层方向代理（例如nginx）。

解释：这个问题用于检查被面试者是否有实际运行Node应用的经验。

问:什么是事件循环

Node只运行在一个单一线程上，至少从Node.js开发者的角度是这样的。在底层， Node是通过libuv来实现多线程的。

Libuv库负责Node API的执行。它将不同的任务分配给不同的线程，形成一个事件循环， 以异步的方式将任务的执行结果返回给V8引擎。可以简单用下面这张图来表示。



每一个I/O都需要一个回调函数——一旦执行完便推到事件循环上用于执行。

解释：这用于检查Node.js的底层知识，例如什么是libuv，它的作用是什么。

问:哪些工具可以用来确保一致性的风格

你可以有如下的工具：

- JSLint
- JSHint
- ESLint
- JSCS- 推荐

在团队开发中，这些工具对于编写代码非常的有帮助，能够帮助强制执行给定的风格指南，并且通过静态分析捕获常见的错误。

解析：用于检查被面试者是否有大型项目开发经验。

问:运算错误与程序员错误的区别

运算错误并不是bug，这是和系统相关的问题，例如请求超时或者硬件故障。而程序员错误就是所谓的bug。

解析：这个题目和Node关系并不大，用于考察面试者的基础知识。

问:为什么npm包管理器有帮助

This command locks down the versions of a package’s dependencies so that you can control exactly which versions of each dependency will be used when your package is installed. – npmjs.com

在你开发Node应用时npm会非常的有用，它可以帮你确定你的依赖的具体的版本号。

解析：它能考察面试者使用npm命令的基础知识和Node.js开发的实际经验。

问:什么是Stub？举个使用场景

Stub是用于模拟一个组件/模块的一个函数或程序。在测试用例中，Stub可以为函数调用提供封装的答案。

当然，你还可以在断言中指明Stub是如何被调用的。

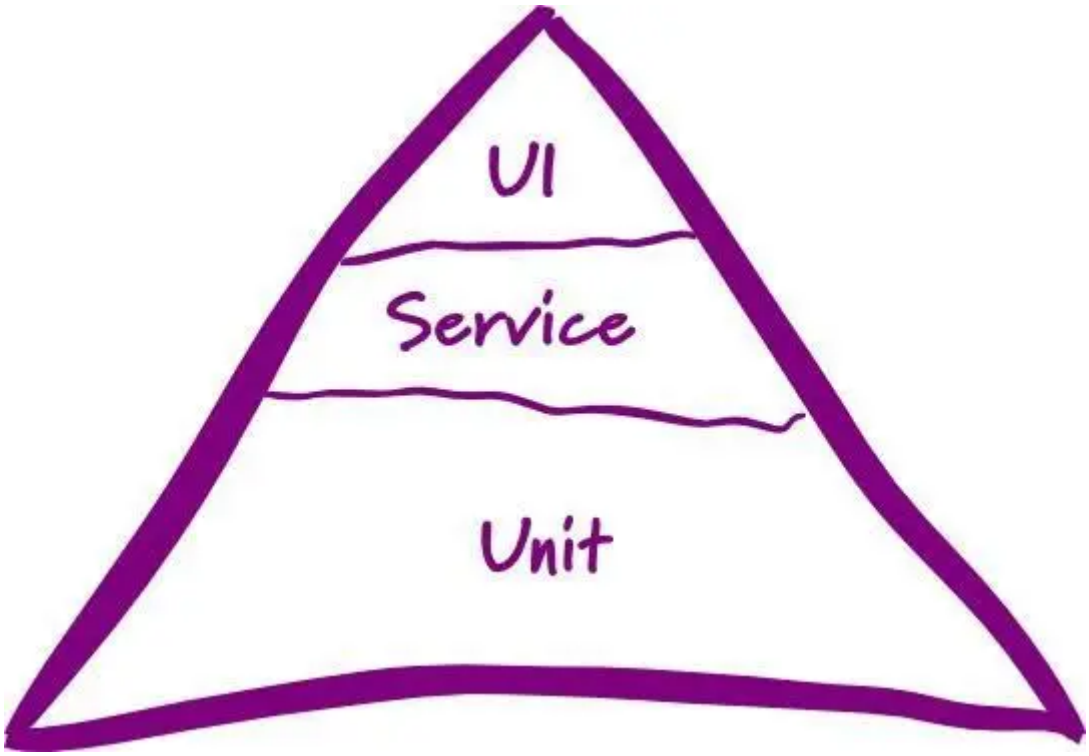
例如在一个读取文件的场景中，当你不想读取一个真正的文件时：

```
var fs = require('fs');

var readFileStub = sinon.stub(fs, 'readFile', function (path, cb) {
    return cb(null, 'filecontent');
});

expect(readFileStub).to.be.called;
readFileStub.restore();

```

解析：用于测试被面试者是否有测试的经验。如果被面试者知道什么是Stub， 那么可以继续问他是如何做单元测试的。

问:什么是测试金字塔？当我们在谈论HTTP API时如何实施它？

测试金字塔指的是： 当我们在编写测试用例时，底层的单元测试应该远比上层的端到端测试要多。



当我们谈到HTTP API时，我们可能会涉及到：

- 有很多针对模型的底层单元测试
- 但你需要测试模型间如何交互时，需要减少集成测试

解析：本文主要考察被面试者的在测试方面的经验。

问:你最喜欢的HTTP框架以及原因

这题没有唯一的答案。本题主要考察被面试者对于他所使用的Node框架的理解程度，考察他是否能够给出选择该框架的理由，优缺点等。